Introducción

El objetivo de este trabajo práctico es desarrollar un servidor web funcional, capaz de recibir y responder solicitudes HTTP para permitir la carga y descarga de archivos a través del navegador gestionando manualmente las solicitudes GET y POST. En este proyecto no sólo pudimos comprender en profundidad los detalles técnicos de la implementación a bajo nivel de un servidor básico con mecanismos extremadamente simples de autenticación y compresión gzip, sino también analizar en detalle su comportamiento mediante diversas experimentaciones.

El trabajo se sitúa dentro de la capa de aplicación del modelo TCP/IP. En esta capa operan los protocolos utilizados por programas que interactúan directamente con la red. El protocolo de aplicación central del servidor es HTTP, que estructura todas las interacciones entre nuestro servidor y el navegador. El mismo se apoya sobre TCP, el protocolo de la capa de transporte encargado de garantizar una transmisión confiable. En este nivel pudimos trabajar con headers, payloads y métodos propios de HTTP y comprender cómo se codifican y procesan las solicitudes reales enviadas desde el navegador web.

Implementación

Funciones del servidor:

imprimir_qr_en_terminal(url)

Primero debíamos familiarizarnos con la biblioteca qrcode provista para realizar la función. Leímos la documentación del sitio *GeeksForGeeks*[1] para entender qué métodos y herramientas incluía dicha biblioteca. Tras leer la documentación, nos quedó muy claro cómo generar un código QR y convertirlo en imagen, pero no sabíamos cómo imprimir en terminal el resultado.

Buscamos en internet y encontramos una librería llamada *Pillow*[2], que podía ayudarnos a imprimir imágenes en la terminal recorriendo los píxeles y usando códigos ANSI de color. Probamos ese enfoque, pero rápidamente notamos que era demasiado verboso y producía un QR muy grande y difícil de leer en algunos casos.

Después de seguir investigando, descubrimos que la biblioteca *qrcode* incluía un método interno llamado *print_ascii()* (gracias a la documentación de *pypi.org*[3]), que permitía imprimir el QR directamente como texto ASCII sin necesidad de procesar la imagen píxel por píxel. Probamos esa alternativa y comprobamos que funcionaba correctamente para múltiples URLs en el archivo "test_qr.py".

Finalmente, nos quedamos con esa solución porque era más simple, más limpia y suficiente para lo que necesitábamos.

[1] https://www.geeksforgeeks.org/python/generate-qr-code-using-qrcode-in-python
[2] https://pillow.readthedocs.io/en/stable
[3] https://pypi.org/project/qrcode

manejar_descarga(archivo, request_line)

La consigna indicaba que, en el modo descarga, el servidor debía responder con un HTTP 200 OK si el archivo solicitado estaba presente y con un HTTP 404 Not Found si no lo estaba. Para poder distinguir estos dos casos necesitábamos una forma de verificar la existencia del archivo dentro de la carpeta correspondiente. Investigando descubrimos que la función *os.path.isfile()* cumplía exactamente esa función. Dado que la librería *os* ya estaba incluida en el código base, asumimos que esa era la herramienta prevista para resolverlo.

Una vez comprobada la existencia del archivo, el siguiente paso era abrirlo y enviar su contenido. Además, la consigna solicitaba devolver tres headers específicos: *Content-Type, Content-Length* y *Content-Disposition*. Buscamos la función de cada uno:

- Content-Type: indica el tipo MIME del archivo a enviar
- Content-Length: especifica el tamaño del contenido en bytes.
- Content-Disposition: define si el contenido debe mostrarse en el navegador o descargarse como archivo adjunto.

Para el Content-Type, recurrimos a la biblioteca mimetypes, mencionada en los recursos[4]. Probamos la función *mimetypes.guess_type()* con el PDF del enunciado usando el archivo test_manejardescarga.py y vimos que funcionaba correctamente.

Para calcular el Content-Length debíamos conocer la cantidad de bytes del archivo. Esto nos recordó, a partir del taller de WebServer de HTTP, que todo lo que se envía por sockets debe estar en formato bytes. Por eso modificamos la apertura del archivo a with open(archivo, "rb") y utilizamos *len()* para obtener la longitud del contenido en bytes.

El header más desconocido para nosotros era Content-Disposition. Sabíamos que indica si el contenido debe verse como página web o descargarse como archivo. Como nuestro servidor actúa como repositorio de archivos, concluimos que siempre debía ser "attachment". Buscando ejemplos en internet encontramos el formato correcto:

Content-Disposition: attachment; filename="nombre_del_archivo.ext"

Para obtener únicamente el nombre del archivo a partir de su path usamos *os.path.basename()*, también consultado en la documentación. Luego simplemente lo convertimos a bytes con *.encode()*.

Con esos tres headers construimos la respuesta del caso exitoso (200 OK). Faltaba implementar el caso donde el archivo no existe. Si bien la consigna solo decía que debía devolverse un 404, decidimos hacerlo de la forma en que lo haría un servidor real: enviando un pequeño HTML con el mensaje “404 Not Found”. En este caso, el Content-Type debía ser "text/html" y el Content-Length debía corresponder al tamaño de ese HTML. Asumimos que el header Content-Disposition no era importante en un caso de error 404.

Nuestra función devuelve siempre los headers concatenados con el contenido, variando según si el archivo existe o no. Probamos ambas situaciones con el PDF del enunciado y verificamos el correcto funcionamiento. Durante las pruebas apareció un error porque intentábamos ejecutar *.encode()* sobre la tupla completa devuelta por *mimetypes.guess_type()*, lo cual solucionamos extrayendo el primer elemento con [0]. Una vez corregido, la función devolvió algo como:

[4] https://docs.python.org/3/library/mimetypes.html

```
b'HTTP/1.1 200 OK\r\n

Content-Type: application/pdf\r\n

Content-Length: 306391\r\n

Content-Disposition: attachment;
filename="TDIV_TP.pdf"\r\n

%PDF-1.4...'
```

Esto confirmaba que la implementación era correcta. También verificamos el funcionamiento del caso de error y funcionó de la manera esperada. Todas estas pruebas se encuentran documentadas en el archivo “test_manejardescarga.py”.

manejar_carga(body, boundary, directorio_destino=".")

Lo primero que tuvimos que hacer fue entender el comportamiento de la función *parsear_multipart(body, boundary)*, encargada de interpretar el mensaje POST enviado por el host al momento de cargar un archivo. Esta función analiza el cuerpo del mensaje y devuelve dos valores: primero el nombre del archivo y luego su contenido en bytes. En caso de no poder extraer correctamente esta información, devuelve un par de valores *None*. Por este motivo, si la función devolvía *None* en alguna de sus dos posiciones, decidimos responder con un HTML de error “Bad Request”, indicando que el servidor no pudo procesar la solicitud correctamente.

Cuando *parsear_multipart* lograba recuperar el archivo exitosamente, el siguiente paso era guardarlo en el directorio indicado por el parámetro directorio_destino. En un primer intento, escribimos:

```
with open(directorio_destino, "wb") as f:

     f.write(contenido_archivo)
```

pero esto generó el error:

IsADirectoryError: [Errno 21] Is a directory

Lo que nos hizo notar que estábamos intentando abrir el directorio, no un archivo dentro de él. Para resolver esto necesitábamos construir la ruta completa al archivo a partir del directorio y del nombre obtenido del POST. Entonces, usamos *os.path.join()*, que justamente permite unir de forma correcta un directorio con un nombre de archivo, independientemente del sistema operativo. Esto nos permitió generar una ruta válida donde guardar el archivo.

Una vez solucionado ese problema, construimos la respuesta HTTP de éxito, enviando un pequeño HTML informando que la carga se realizó correctamente junto con los headers correspondientes.

Finalmente, probamos la función en test_manejarcarga.py utilizando un archivo de texto simple llamado primeracarga.txt. Creamos la carpeta archivos_servidor y verificamos que el archivo efectivamente se guardara dentro de ella. El comportamiento fue el esperado.

start_server(archivo_descarga=None, modo_upload=False)

Antes de implementar el servidor, necesitábamos repasar el manejo de sockets visto en el taller de WebServer, por lo que volvimos a consultar la documentación (especialmente la guía de Real Python sobre Socket Programming[5]). Con esa base, lo primero que hicimos fue obtener la IP del servidor mediante *get_wifi_ip()*. Luego creamos el server_socket utilizando dicha IP y permitiendo que el sistema operativo asignara automáticamente un puerto disponible. Una vez configurado el socket, imprimimos la URL final y el código QR de la misma.

El establecimiento de la conexión y la recepción del request fueron relativamente sencillos. Decidimos recibir un máximo de 1024 bytes, ya que esa era la cantidad estándar utilizada en el material de Real Python, pero luego lo cambiamos a un número mayor (4096) para evitar posibles problemas por no poder recibir requests más largos. La mayor dificultad apareció al intentar decodificar la solicitud HTTP. Nos guiamos por la función *parsear_multipart()*, que utilizaba split con separadores \r\n. Aplicando esa misma estrategia, dividimos el request en líneas y tomamos la primera, ya que de allí debíamos obtener el método y la ruta. Para eso volvimos a usar split, pero esta vez con el espacio " " como separador.

Una vez recuperados el método y la ruta, debíamos generar la respuesta correcta según el modo del servidor. Para un método GET, la ruta "/" indicaba que debíamos servir la página principal (distinta según fuera modo upload o download). En cambio, la ruta "/download" correspondía a la solicitud directa de un archivo, para lo cual debíamos invocar la función manejar_descarga(). En ambos casos generamos la página HTML correspondiente mediante la función auxiliar provista y enviamos la respuesta HTTP adecuada.

Para el método POST, en cambio, no alcanzaba con leer la primera línea: debíamos procesar tanto los headers como el body. En particular, necesitábamos extraer el boundary, que es fundamental para separar las partes del contenido multipart/form-data. Sabíamos que el boundary aparece luego del texto "boundary=", por lo que buscamos la línea que lo contuviera, aplicamos .split() y nos quedamos con la parte inmediatamente posterior. Una vez obtenido el boundary, ya podíamos llamar a la función manejar_carga() para guardar el archivo.

Con todos los casos cubiertos, lo único que quedaba era cerrar la conexión. Sin embargo, durante las primeras pruebas en ambos modos descubrimos un problema importante: apenas escaneábamos el QR, la conexión se cerraba inmediatamente. Esto sucedía porque habíamos implementado el servidor de modo que cerrara el socket del servidor luego de cualquier request, incluso luego del primer GET / que sirve la página HTML. Esto era incorrecto, ya que impedía que el cliente pudiera hacer después la request real del archivo: el GET /download o el POST de upload.

Entendimos entonces que el socket del servidor sólo debía cerrarse después de una operación efectiva de carga o descarga, es decir, después de un request "/download" válido (if metodo == "GET" and ruta == "/download" and archivo_descarga is not None) o de un POST en modo upload (if metodo == "POST" and modo_upload). Para lograr este comportamiento incluimos la aceptación de conexiones

[5] https://realpython.com/python-sockets

dentro de un ciclo infinito while True, que únicamente se interrumpe con un break en esos dos casos específicos.

Tras implementar esta modificación, el servidor comenzó a funcionar correctamente en ambos modos.

Implementación de compresión *gzip*:

En primer lugar, leímos la documentación de *gzip* incluida en las consignas[6]. Luego, comenzamos por pensar cómo podríamos detectar si el cliente pone en el header de su request "Accept-Encoding: gzip". Para ello primero imprimimos unas requests con el objetivo de ver exactamente cómo se construían. Concluimos que lo más sencillo era recorrer la request splitteando por cada "/r/n" y chequeando si los primeros 17 caracteres de esa línea eran justamente "Accept-Encoding:" y si dicha línea incluía la palabra "gzip". En ese caso, le asignamos a la variable booleana "gzip_accept" True (si no por defecto la asignamos a False).

A continuación, configuramos la opción por línea de comandos para activar/desactivar la compresión. Para informarnos sobre cómo realizar esto leímos dos artículos de python.org[7] que encontramos al buscar en Google "configuración de línea de comandos". Allí encontramos que la función *sys.argv* devuelve una lista de los argumentos que se pasan por línea de comandos del script de Python. De esta forma, para entender mejor su funcionamiento, *printeamos* varios *sys.argv* usando distintos argumentos. Así, interpretamos que para abordar la consigna era necesario fijarse si la opción de comprimir estaba dentro de los parámetros de la línea enviada por consola al iniciar el servidor en modo descarga. Por lo tanto, definimos a este parámetro como "--comprimir" y en caso de que éste se halle en la línea le asignamos a la variable booleana "gzip_activo" True (si no por defecto la asignamos a False). Luego, pasamos como nuevo parámetro para la función *manejar_descarga* el *Bool* resultante de hacer un *and* lógico entre "gzip_accept" y "gzip_activo".

Una vez hecho esto, incluimos el header "Content-Encoding: gzip" en las responses que tuviesen compresión. Para ello revisamos un artículo de Mozilla Developer Network[8] y sencillamente creamos un string de nombre "header_content_encoding" y le asignamos una string de bytes vacía, en la función *manejar_descarga*. Luego, si el *Bool* pasado como tercer parámetro de la función es True, se le asigna a esta variable el string de bytes correspondiente, "Content-Encoding: gzip\r\n". Esta porción del header es añadida al header total de la misma forma que antes.

Por último, implementamos la compresión de archivos en sí misma. Para ello, leímos la documentación sobre *gzip* incluida en la consigna[9]. En ese momento, pensamos si la compresión del archivo debía realizarse en el programa creando un archivo en el disco o no. Un ejemplo proveído en la página de la documentación hacía justamente esto. Sin embargo, consideramos que por un lado, no es práctico crear archivos nuevos comprimidos cada vez que se solicite una descarga, y además ya teniendo implementada la lectura del contenido en el programa, lo más simple es comprimir este directamente. Para ello usamos la función *gzip.compress* que toma un string de bytes y devuelve un objeto de bytes con los datos ingresados comprimidos. Entonces, simplemente en la misma sección en la que nos fijamos si el bool pasado como tercer parámetro de la función es True para modificar el header, en caso

[6] https://docs.python.org/3/library/gzip.html
[7] 1. Línea de comandos y entorno — documentación de Python - 3.10.19 y sys — Parámetros y funciones específicos del sistema — documentación de Python - 3.10.19.
[8] https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Headers/Content-Encoding
[9] https://docs.python.org/3/library/gzip.html

de que se cumpla la guarda del condicional, comprimimos con *gzip.compress* el contenido leído desde la ruta de la descarga.

Una vez ya realizado lo descrito, decidimos probar si funcionaba la implementación. Esta fue la parte más compleja de esta sección del trabajo práctico. Inicialmente, cuando realizamos las descargas de los archivos que veníamos utilizando para las pruebas todos se descargaban en la misma forma que antes sin compresión aparente. Por un lado, procedimos a chequear si el header modificado se estaba enviando correctamente[10]. Para esto, realizamos una descarga desde el navegador web pero antes abrimos la sección *Network* de las *DevTools* de Chrome. Allí vimos los *response-headers* y comprobamos que todo estaba funcionando correctamente. De todos modos, queríamos más pruebas de que la respuesta HTTP estaba utilizando verdaderamente *gzip*. Entonces, decidimos buscar exactamente cómo se manifestaba la compresión. En una pregunta de la página *StackOverFlow,* un usuario detalla que la compresión usando *gzip* en HTTP es transparente, el usuario no debería ver expresamente su actuar[11]. La idea general es que simplemente debe comprimirse el archivo antes de enviarse así el transporte del contenido es más rápido porque es menos costoso en términos de ancho de banda. Entonces, es normal que el archivo se descargue como "si no hubiese compresión" porque el navegador se ocupa de descomprimir el archivo antes de entregarlo. Sin embargo, esto tampoco nos pareció suficiente para considerar que la implementación funcionaba. De este modo, se nos ocurrió buscar cuales eran los *magic numbers* que identificaban un archivo comprimido con *gzip.* La secuencia de caracteres hexadecimales es "1F 8B"[12]. Así, nos propusimos ver si había alguna forma de descargar el contenido sin descomprimirlo inmediatamente. Como el navegador lo realizaba inmediatamente, evidentemente había que realizar la descarga por fuera de nuestros navegadores estándar. Buscando si existía algún comando que permitiese hacer esto por consola encontramos el *curl*[13]. Al principio, utilizando este comando el archivo no se comprimía, pero *debuggeando* el código encontramos que esto se debía a que powershell por defecto no estaba mandando "Accept-Encoding: gzip" en la *request*. Entonces lo agregamos manualmente al header (con -H) y guardamos el contenido descargado en un archivo. Así, luego usamos *Format-Hex*[14] para revisar los bytes del archivo y comprobamos que los *magic numbers* estaban presentes. Realizamos este mismo procedimiento con varios archivos. Posteriormente, reparamos en que podíamos también capturar los paquetes con el wireshark y verificar la presencia de los números. Este chequeo funcionó correctamente.

```
PS C:\Users\cande> curl.exe -O http://192.168.1.56:54142/download -H "Accept
-Encoding: gzip"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curr
ent
                                 Dload  Upload   Total   Spent    Left  Spee
d
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--
  0     0    0     0    0     0      0      0 --:--:--  0:00:01 --:--:--
  0     0    0     0    0     0      0      0 --:--:--  0:00:02 --:--:--
  0     0    0     0    0     0      0      0 --:--:--  0:00:03 --:--:--
 40 40.1M   40 16.0M    0     0  3979k      0  0:00:10  0:00:04  0:00:06 397
 95 40.1M   95 38.1M    0     0  7614k      0  0:00:05  0:00:05 --:--:-- 761
100 40.1M  100 40.1M    0     0  7856k      0  0:00:05  0:00:05 --:--:-- 990
PS C:\Users\cande> Format-Hex .\download -Count 48

   Label: C:\Users\cande\download

          Offset Bytes                                           Ascii
                 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
          ------ -----------------------------------------------  -----
0000000000000000 1F 8B 08 00 1B D6 23 69 02 FF 54 BA 07 50 53 51  ♦♦♦ ♦Ö#i♦_
0000000000000010 D7 36 1A BA 74 91 2E 25 22 8A F4 2E 4D 20 80 20  ×6♦°t♦.%"_
0000000000000020 22 20 D2 7B EF D2 A4 77 42 11 90 AE 14 E9 BD 49  " Ò{ïÒ¤wB_
```

[10] Esto sugería hacer un usuario de *Quora* para ver si la compresión *gzip* se aplicaba en un sitio web: https://www.quora.com/Why-some-GZIP-tests-show-compression-enabled-on-my-site-and-others-show-that-is-not.

[11] "What is gZip compression?" (https://stackoverflow.com/questions/16691506/what-is-gzip-compression).

[12] https://www.rfc-editor.org/rfc/rfc1952

[13] https://curl.se/docs/tutorial.html

[14] https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/format-hex?

Experimentación

Experimento 1

Para comenzar los experimentos con la compresión *gzip*, comparamos los tiempos de transferencia y tamaños de archivo para imágenes y documentos de texto. Para hacer esto se nos ocurrió utilizar el módulo time de python que ya nos era familiar. En su documentación[15] encontramos una función llamada *time.perf_counter_ns()* que sirve para medir en nanosegundos la diferencia entre dos tiempos, es decir, solo es válida utilizando la diferencia entre dos de sus resultados. De este modo, colocamos en el código un timer inmediatamente después de recibir la *request,* otro inmediatamente después de terminar de enviar los bytes de la respuesta, e imprimimos su diferencia por terminal.

Tipo	Tamaño		Ratio de compresión (%)	Tiempo de transferencia	
	sin gzip (Bytes)	con gzip (Bytes)		sin gzip (nanosegundos)	con gzip (nanosegundos)
.PDF	13.608.013 B	13.469.219 B	1.0199%	915.387.600 ns	1.462.440.600 ns
.PDF	3.616.600 B	2.277.869 B	37.0163%	338.647.400 ns	731.126.400 ns
.PDF	51.248 B	47.345 B	7.6159%	328.397.600 ns	377.649.300 ns
.CR2	23.582.391 B	23.147.903 B	1.8424%	393.270.200 ns	2.080.416.500 ns
.PNG	4.390.143 B	4.235.344 B	3.5261%	372.619.800 ns	752.295.900 ns
.PNG	91.663 B	77.296 B	15.6737%	317.784.600 ns	330.514.900 ns

Tabla I *: Resultados del Experimento 1.*

En general, analizando la Tabla I, notamos que el ratio de compresión era relativamente bajo (excepto en el pdf de tamaño mediano y el png pequeño) y el tiempo de transferencia era mucho mayor cuando se utilizaba la compresión *gzip* que cuando se enviaba el archivo original. Esto nos sorprendió ya que, como el *gzip* disminuye el tamaño del archivo, el tiempo de transmisión debería ser menor y en consecuencia uno esperaría que el tiempo de transferencia total disminuya. Esta inesperada diferencia en los tiempos de transferencia puede observarse en el gráfico a continuación:

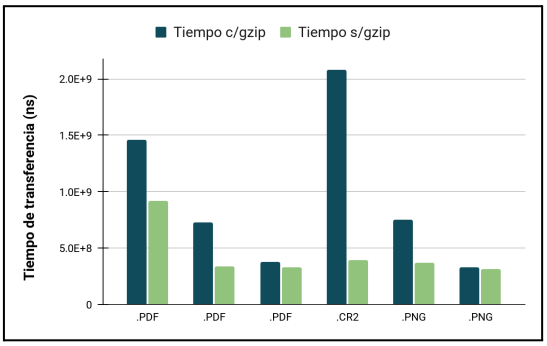

Figura I*: Tiempos de transferencia con y sin compresión gzip de los archivos del Experimento 1.*

Como se observa en la figura I, en todos los casos el tiempo de transferencia utilizando *gzip* fue mayor que sin hacerlo. En particular, en algunos casos el tiempo utilizando la compresión fue más del doble del original.

[15] https://docs.python.org/es/3.10/library/time.html

Experimento 2

Investigando, reparamos en que hay un *trade-off* entre costo a nivel de CPU y nivel de compresión[16]. Esto quiere decir que a mayor nivel de compresión, dependiendo de las capacidades de cómputo del servidor (en este caso nuestras computadoras personales), también la función *manejar_descarga* tardará más en ejecutarse. En consecuencia, los resultados de los experimentos ya no son tan sorprendentes. En este sentido, el nivel de compresión en la implementación de *gzip* del módulo de python, a menos que se indique otro, es el más alto (9). Además, para este momento reparamos en que los formatos de archivo con los que realizamos los experimentos ya incluyen algún tipo de compresión interna. Entonces, los ratios de compresión bajos pueden deberse a que, como *gzip* comprime sin pérdidas (es decir, la longitud media del código está acotada por la entropía), no hay lugar para demasiada compresión en un archivo ya comprimido[17]. En este sentido, ningún compresor sin pérdidas puede lograr que el ratio entre la cantidad promedio de dígitos binarios por palabra en el código y la entropía sea menor a 1. Para comprobar si nuestras explicaciones para los tiempos de transferencia y tamaños de archivo tenían sentido, buscamos cómo descomprimir un .PDF. Logramos hacerlo siguiendo las instrucciones que encontramos en un foro de *Stack Exchange*[18]. Así, decidimos hacer un segundo experimento, con el siguiente procedimiento: por cada descarga del pdf descomprimido variamos en el código el nivel de compresión del *gzip* modificando el parámetro *compresslevel*[19] y registramos el tamaño del archivo comprimido y el tiempo de transferencia.

Tamaño original:	Tamaño (Bytes)	Tiempo de transferencia (segundos)	compress level	Ratio de compresión (%)	Variación en el tiempo
106,148833 MB	12.273.242 B	46,32 s	9	88,44%	+39,45 s
Tiempo sin gzip:	13.112.970 B	4,92 s	6	87,47%	-1,94 s
	14.664.237 B	3,13 s	5	86,18%	-3,74 s
6,86 s	15.872.272 B	2,30 s	4	85,05%	-4,56 s

Tabla II *: Resultados del Experimento 2.*[20]

Como se observa en la Tabla II, a medida que *compresslevel* disminuye, el tiempo de transferencia baja. Esto probablemente se deba a que la función *manejar_descarga* tarda menos en ejecutarse, es decir, el costo a nivel CPU es menor. Además, si bien el ratio de compresión crece conforme lo hace el nivel de compresión, hay aproximadamente 3,6 MB de diferencia entre el tamaño del archivo más comprimido y el menos comprimido de la tabla. Esto representa tan solo un 3,39% del tamaño original del archivo. Asimismo, el tiempo de transferencia se reduce en aproximadamente 44,02 segundos entre estos dos niveles de compresión. Por lo tanto resulta más óptimo bajar el nivel de compresión a algo menor que 9. Consideramos que, en última instancia, la compresión tiene como uno de sus grandes objetivos que la transmisión sea más rápida y en consecuencia el envío también. En este sentido, encontramos que

[16] https://www.pingdom.com/blog/can-gzip-compression-really-improve-web-performance/
[17] https://stackoverflow.com/questions/10547890/are-all-pdf-files-compressed
[18] https://unix.stackexchange.com/questions/17663/how-to-know-if-a-pdf-file-is-compressed-or-not
[19] Este parámetro indica el nivel de compresión del archivo: va de 0 a 9, donde los valores bajos producen menor compresión y los valores altos mayor compresión.
[20] Nótese que los tiempos de transferencia fueron tomados de la misma forma que en el primer experimento. En este caso, decidimos aproximar los tiempos y pasarlos a segundos para que su análisis fuese visualmente más evidente. Esta aproximación no modifica las conclusiones que se desprenden del experimento en ninguno de los ensayos.

compresslevel = 6 es el default en la documentación de *gzip* del proyecto GNU[21], por lo tanto decidimos dejar especificado este nivel en el código.

Seguridad

La manera en la que está construido nuestro servidor lo deja expuesto a varios problemas de seguridad importantes. Esto se debe principalmente a que toda la comunicación se hace en texto plano, sin usar ningún tipo de cifrado, ni certificados, ni mecanismos que permitan identificar si realmente estamos hablando con el servidor correcto. Por esa razón, un atacante conectado a la misma red que nosotros podría comprometer la confidencialidad, la integridad o la disponibilidad del sistema sin grandes dificultades. A continuación explicamos tres amenazas concretas (una para cada componente de seguridad previamente mencionado) junto con las medidas teóricas que serían necesarias para solucionarlas.

Confidencialidad - Sniffing:

El primer problema aparece en la confidencialidad. Nuestro servidor envía todo en texto plano: los headers, las rutas y hasta los archivos completos que se suben o descargan. Como usamos un socket TCP común, nada de lo transmitido está cifrado. Esta decisión funciona para una demostración académica, pero en una red Wi-Fi real, cualquier persona conectada a la misma red puede interceptar los paquetes utilizando herramientas de sniffing. Un sniffer básicamente captura el tráfico que viaja por la red y lo muestra tal cual llega. De este modo, como no hay criptografía, el atacante vería literalmente toda la comunicación. Una forma convencional de protegerse de esto es usar TLS, un protocolo que cifra toda la comunicación. TLS funciona con un proceso de "handshake": el cliente envía un mensaje inicial (Client Hello) diciendo qué algoritmos soporta y el servidor responde con un "Server Hello" más un certificado digital que contiene su clave pública. El cliente verifica ese certificado para asegurarse de que es auténtico y no está vencido, y luego ambos generan en conjunto una clave simétrica (por ejemplo, mediante el algoritmo RSA o el Diffie-Hellman) Finalmente, toda la comunicación después de ese punto se cifra con un algoritmo simétrico como AES. Si aplicáramos esto a nuestro servidor, habría que reemplazar el socket TCP común por uno que inicie un canal TLS y conseguir un certificado válido para nuestro servidor. Esto cambiaría por completo el intercambio de mensajes porque, antes de recibir cualquier GET o POST, ocurriría el intercambio de Client Hello, Server Hello, certificado y generación de claves. Recién después de ese proceso se enviarían las requests HTTP.

Integridad - Spoofing:

El segundo problema está relacionado con la integridad. Actualmente el cliente no tiene forma de comprobar si realmente se está conectando a nuestro servidor o si alguien se está haciendo pasar por él. Esto habilita un ataque mediante ARP spoofing. ARP es el protocolo que usan las computadoras de una red local para averiguar cuál es la dirección MAC asociada a una dirección IP. El inconveniente es que ARP no tiene ningún mecanismo para verificar si los mensajes que recibe son legítimos. Un atacante como Trudy puede enviar mensajes ARP falsos diciendo "yo soy la IP del servidor" cuando realmente no lo es, y convencer a las víctimas de enviarle el tráfico a él. También puede hacer lo mismo en dirección contraria, haciéndole creer al servidor que su máquina es la del cliente. Así el atacante se coloca en el medio ("man-in-the-middle") y puede modificar archivos, reemplazarlos o incluso devolver un archivo distinto sin que el cliente lo note. Nuestro servidor es vulnerable porque no ofrece ningún tipo de autenticación. Una manera de mitigarlo sería justamente aprovechar la autenticación

[21] https://www.gnu.org/software/gzip/manual/gzip.html

que provee TLS mediante certificados. Cuando el servidor envía su certificado digital en el handshake, el cliente puede verificar que la identidad coincide con la de un servidor confiable. Además de eso podríamos sumar verificación de integridad de los archivos, por ejemplo enviando un hash criptográfico (como un SHA-256) junto con cada descarga. El cliente compararía ese hash con el que calcula localmente y sabría si el archivo fue alterado durante el camino. En la práctica, esto significaría que después de que el servidor envía un archivo, tendría que enviar también el hash, y el cliente lo verificaría antes de aceptarlo.

Disponibilidad - Ataque DoS (Denial of Service):

El tercer aspecto a analizar es la disponibilidad. Nuestro servidor atiende solo una conexión y además funciona de manera bloqueante, lo que significa que queda esperando indefinidamente a que el cliente envíe datos. Un atacante como Trudy podría explotar esto para hacer un ataque de denegación de servicio (DoS). Por ejemplo, puede conectarse al servidor y no enviar nada, manteniendo la conexión abierta. Como el servidor no tiene límites de tiempo ni de conexiones, queda completamente bloqueado y no puede atender a nadie más. También podría abrir repetidamente muchas conexiones para mantenerlo ocupado indefinidamente. Este problema surge porque no implementamos mecanismos básicos como timeouts, límites de uso por IP o filtrado de tráfico. Una solución sería agregar un tiempo máximo de espera después del *client_socket.accept()*. Si pasa cierto tiempo sin que llegue una request válida, el servidor debería cortar la conexión. También serviría limitar cuántas conexiones puede abrir la misma IP en un intervalo breve. Finalmente, se podría usar un firewall o un mecanismo de IDS/IPS que detecte patrones de tráfico anómalos y bloquee automáticamente direcciones sospechosas. En términos del intercambio de mensajes, estas soluciones añadidas modificarían lo que ocurre entre aceptar la conexión y recibir la request HTTP, porque ahora habría controles adicionales antes de procesar cualquier GET o POST.

En resumen, nuestro servidor es funcional pero inseguro en varios niveles. La falta de cifrado permite que cualquiera capture los archivos transmitidos, la falta de autenticación habilita ataques como ARP spoofing que comprometen la integridad, y la ausencia de medidas de control permite atacar su disponibilidad mediante DoS.

Bonus

Al final del trabajo decidimos intentar el ejercicio bonus, que consistía en agregarle al servidor un pequeño mecanismo de autenticación para que solo usuarios autorizados puedan subir o descargar archivos. La forma más simple de hacerlo fue definir una contraseña fija dentro del código (en la línea 12: contraseña = "DINASTIA") y pedirle al usuario que la ingrese antes de permitirle continuar.

Lo primero que hicimos fue agregar una nueva página HTML que ahora aparece apenas el cliente accede a la URL del QR (a partir de la función *generar_html_password*). Esta página contiene un formulario muy sencillo donde el usuario escribe la contraseña. Cuando este envía el formulario, el navegador hace un POST hacia una nueva ruta que agregamos nosotros, que se llama "/autenticacion". En esa ruta el servidor recibe los datos del formulario y los compara con la contraseña que definimos en el código (en *start_server*, esto sucede para el caso if metodo == "POST" and ruta == "/autenticacion") . Si no coinciden, el servidor devuelve una página diciendo que la contraseña es incorrecta. Solo en caso de que coincidan, le mostramos al usuario la interfaz original del servidor, que puede ser la de descarga o la de subida dependiendo del modo en el que ejecutamos el programa.

Para poder leer correctamente lo que enviaba el formulario, tuvimos que usar el módulo *urllib.parse* de Python. Lo incluimos porque los formularios web envían los datos en un formato llamado

application/x-www-form-urlencoded, donde los pares clave-valor llegan como un texto del estilo "contra=utdt2025". Este texto no siempre llega de la misma forma, puede venir mezclado con símbolos especiales o con codificaciones propias del navegador. Por eso usamos la función *parse_qs*, que sirve para interpretar ese tipo de cadenas y convertirlas en un diccionario manejable desde Python. Por ejemplo, si el body del POST es contra=utdt2025, entonces *parse_qs* lo transforma automáticamente en {"contra": ["utdt2025"]}. En nuestro caso usamos *params.get("contra", [""])[0]* para extraer el valor real de la contraseña enviada por el usuario de manera segura, incluso si por algún motivo la clave no estuviera presente en el formulario.

Tras hacer esto, todo funcionaba bien para la contraseña, pero tuvimos un problema cuando intentamos subir un archivo. Después de autenticarse correctamente, el servidor enviaba la pantalla de subida, pero cuando tratábamos de subir un archivo el servidor volvía a mostrar la página de "contraseña incorrecta". Sospechamos que si estaba ocurriendo esto era porque el servidor estaba interpretando la subida del archivo como un request POST acompañado por la ruta "/autenticacion". Esto pasaba porque el formulario de upload que ya teníamos no incluía ningún atributo *action*, así que el navegador, por defecto, mandaba el archivo a la misma ruta desde la que se había cargado la página. Como esa ruta era "/autenticacion", el servidor interpretaba la subida de archivos como si fuese otro intento de autenticación y obviamente la validación fallaba. Para resolverlo alcanzó con agregar action="/upload" en el formulario de subida. De esa manera, el navegador envía el archivo a la ruta correcta y el servidor lo procesa como siempre. No hizo falta cambiar nada del manejo interno de subida o descarga.

Este mecanismo de autenticación es básico y no tiene nada de criptografía real ni manejo de sesiones, pero cumple con lo que pedía el bonus y nos permitió practicar varias cosas a la vez: cómo manejar rutas nuevas en un servidor, cómo procesar formularios HTML, cómo usar *parse_qs* para interpretar lo que envía un navegador y también cómo detectar errores que aparecen simplemente por la forma en que el navegador decide a qué URL realizar los envíos.

Análisis de protocolos y headers HTTP

Metodología de captura:

Con la finalidad de analizar el comportamiento del servidor utilizamos Wireshark para capturar el tráfico HTTP entre el navegador y la máquina donde corrimos el servidor. Primero ejecutamos el servidor en modo descarga y, una vez generado el código QR, lo escaneamos con un teléfono celular y abrimos la URL desde el navegador. En Wireshark filtramos los paquetes por dirección IP del servidor y protocolo HTTP usando el filtro tcp.port == [*puerto arrojado en la terminal*]. De esta forma, ya que el puerto aparece o en el origen o en el destino, pudimos aislar las secuencias de mensajes correspondientes a la descarga, y luego reconstruir cada intercambio usando la opción "Follow TCP stream". Repetimos el proceso para el modo carga de archivos.

Observamos que antes de cualquier tráfico HTTP se produce el 3-way handshake característico de TCP: (1) SYN (cliente → servidor) (2) SYN, ACK (servidor → cliente) (3) ACK (cliente → servidor)

No.	Time	Source	Destination	Proto	Leng	Info
161	18.538090	192.168.0.243	192.168.0.184	TCP	78	56823 → 63805 [SYN] Seq=0
162	18.538330	192.168.0.184	192.168.0.243	TCP	74	63805 → 56823 [SYN, ACK]
163	18.577972	192.168.0.243	192.168.0.184	TCP	66	56823 → 63805 [ACK] Seq=1

Caso de descarga de archivo (GET)

Después del establecimiento de la conexión TCP (3-way handshake), el navegador del cliente envía la primera solicitud HTTP al servidor. En Wireshark esta comunicación se ve de la siguiente manera:

```
164 18.629648 192.168.0.243 192.168.0.184 HTTP 473 GET / HTTP/1.1
165 18.629979 192.168.0.184 192.168.0.243 HTTP 465 HTTP/1.1 200 OK  (text/html)
```

La solicitud es un GET / HTTP/1.1, donde el cliente solicita la página principal del servidor. HTTP/1.1 200 OK indica que la solicitud se procesó correctamente. El servidor devolvió un documento HTML, especificado por el header Content-Type: text/html, y declaró el tamaño del contenido con Content-Length: 334. El payload que sigue a estos headers corresponde al HTML generado por el servidor para la página inicial.

Después de la primera carga, el navegador Safari envió automáticamente una request adicional: GET /favicon.ico HTTP/1.1. En el proceso de interpretar los paquetes aprendimos que este archivo es el ícono pequeño que los navegadores muestran en la pestaña, pero nuestro servidor no lo implementa, por lo que respondió con: HTTP/1.1 404 Not Found.

```
173 18.904579 192.168.0.243 192.168.0.184 HTTP 432 GET /favicon.ico HTTP/1.1
174 18.905139 192.168.0.184 192.168.0.243 HTTP 159 HTTP/1.1 404 Not Found  (text/html)
```

Luego, el navegador envió un POST a /autenticacion utilizando application/x-www-form-urlencoded, con el payload del formulario donde se incluye la contraseña. Wireshark muestra este contenido reensamblado a partir de varios segmentos TCP. El servidor respondió con un HTTP/1.1 200 OK, indicando que la autenticación fue correcta y devolviendo la interfaz HTML correspondiente.

```
532 29.135485 192.168.0.243 192.168.0.184 HTTP  81 POST /autenticacion HTTP/1.1  (application/x-www-form-urlencoded)
533 29.135749 192.168.0.184 192.168.0.243 TCP   66 63805 → 56828 [ACK] Seq=1 Ack=580 Win=64768 Len=0 TSval=78000998 TSecr=660564242
534 29.136411 192.168.0.184 192.168.0.243 HTTP 611 HTTP/1.1 200 OK  (text/html)
```

En el paquete de autenticación, Wireshark muestra directamente la contraseña dentro del cuerpo del POST porque toda la comunicación viaja en texto plano. Esto se debe a que, como no usamos TLS, los datos del formulario (incluida la clave) aparecen sin ningún tipo de codificación ni cifrado. Así, se confirma lo discutido en la sección de seguridad: en una red Wi-Fi real, cualquier sniffer, haciendo lo mismo que nosotros, podría capturar el tráfico y ver las credenciales tal cual fueron enviadas.

```
▼ HTML Form URL Encoded: application/x-www-form-urlencoded
  ▼ Form item: "contra" = "DINASTIA"
      Key: contra
      Value: DINASTIA
```

Después de la autenticación, el cliente envía la solicitud GET /download y el servidor comienza la transferencia del archivo elegido. En este caso, se trata de una imagen, por lo que la respuesta del servidor incluye un HTTP/1.1 200 OK (PNG) seguido del payload correspondiente. A partir de ese punto, Wireshark muestra la secuencia de segmentos TCP que contienen los fragmentos del archivo.

```
558 31.306295 192.168.0.243 192.168.0.184 HTTP 532 GET /download HTTP/1.1
559 31.347733 192.168.0.184 192.168.0.243 TCP   66 63805 → 56829 [ACK] Seq=1 Ack=467 Win=65024 Len=0 TSval=78003210 TSecr=1693262123
560 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=1 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
561 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=1449 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
562 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=2897 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
563 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=4345 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
564 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=5793 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
565 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=7241 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
566 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=8689 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
567 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=10137 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
568 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=11585 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
569 31.352793 192.168.0.184 192.168.0.243 TCP  15… 63805 → 56829 [ACK] Seq=13033 Ack=467 Win=65024 Len=1448 TSval=78003215 TSecr=1693262123 [TCP PDU reassembled in 628]
```

Luego de que el servidor termina de enviar el archivo, se observa en Wireshark la respuesta final HTTP/1.1 200 OK, seguida de varios paquetes TCP sin payload. Estos intercambios corresponden al mantenimiento y cierre de la conexión: aparecen ACKs normales, actualizaciones de ventana (*TCP Window Update*) y mensajes de keep-alive, que TCP utiliza para confirmar que el canal sigue activo aunque ya no se estén transmitiendo datos. Finalmente, el servidor envía un FIN, ACK, iniciando el cierre ordenado de la sesión, y el cliente responde con el ACK correspondiente. Nos llamó la atención que se necesitara solamente un FIN para cerrar la conexión. Consultamos en la clase práctica y nos fue explicado que es posible que algunas conexiones se cierren con el FIN del lado de un solo agente, mientras que el otro espera un timeout para cerrarla. Asumimos que esto puede servir para ser más eficientes en términos de congestión de la red.

628	31.374206	192.168.0.184	192.168.0.243	HTTP	635	HTTP/1.1 200 OK (PNG)
629	31.376619	192.168.0.243	192.168.0.184	TCP	66	56829 → 63805 [ACK] Seq=467 Ack=86881 Win=59008 Len=0 TSval=1693262193 TSecr=78003234
630	31.377756	192.168.0.243	192.168.0.184	TCP	66	56829 → 63805 [ACK] Seq=467 Ack=91795 Win=54208 Len=0 TSval=1693262193 TSecr=78003237
631	31.377785	192.168.0.243	192.168.0.184	TCP	66	[TCP Window Update] 56829 → 63805 [ACK] Seq=467 Ack=91795 Win=131072 Len=0 TSval=1693262193 TSecr=78003237
632	31.630303	192.168.0.243	192.168.0.184	TCP	60	[TCP Keep-Alive] 56829 → 63805 [ACK] Seq=466 Ack=91795 Win=131072 Len=0
633	31.630364	192.168.0.184	192.168.0.243	TCP	66	[TCP Keep-Alive ACK] 63805 → 56829 [ACK] Seq=91795 Ack=467 Win=65024 Len=0 TSval=78003493 TSecr=1693262193
636	32.808761	192.168.0.243	192.168.0.184	TCP	66	56829 → 63805 [FIN, ACK] Seq=467 Ack=91795 Win=131072 Len=0 TSval=1693263622 TSecr=78003493
637	32.808865	192.168.0.184	192.168.0.243	TCP	66	63805 → 56829 [ACK] Seq=91795 Ack=468 Win=65024 Len=0 TSval=78004671 TSecr=1693263622

Caso de carga de archivo (POST):

En esta parte de la traza se observa nuevamente el ciclo normal de solicitud y respuesta en HTTP: primero se envía un GET /, el servidor responde con un HTTP 200 OK, y luego el navegador intenta cargar automáticamente el ícono del sitio mediante un GET /favicon.ico, al que el servidor contesta con un 404 Not Found, igual que describimos anteriormente. Esta secuencia muestra la secuencia normal de solicitud y respuesta en HTTP.

El upload comienza sólo después de este intercambio inicial: el navegador establece una nueva conexión, envía el POST /autenticacion y, solo si la contraseña es aceptada, procede a transmitir el archivo en múltiples segmentos TCP. A partir de este punto es donde analizamos en detalle cómo se desarrolla la transferencia de subida.

423	21.033793	192.168.0.243	192.168.0.184	TCP	78	56856 → 55603 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=677673867 TSecr=0 SACK_PERM
424	21.033980	192.168.0.184	192.168.0.243	TCP	74	55603 → 56856 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=84950971 TSecr=677673867
425	21.037538	192.168.0.243	192.168.0.184	TCP	66	56856 → 55603 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=677673874 TSecr=84950971
430	21.395360	192.168.0.243	192.168.0.184	HTTP	473	GET / HTTP/1.1
431	21.395730	192.168.0.184	192.168.0.243	HTTP	465	HTTP/1.1 200 OK (text/html)
432	21.395846	192.168.0.184	192.168.0.243	TCP	66	55603 → 56856 [FIN, ACK] Seq=400 Ack=408 Win=65024 Len=0 TSval=84951333 TSecr=677674232
433	21.401332	192.168.0.243	192.168.0.184	TCP	66	56856 → 55603 [ACK] Seq=408 Ack=401 Win=131392 Len=0 TSval=677674238 TSecr=84951333
434	21.412129	192.168.0.243	192.168.0.184	TCP	66	56856 → 55603 [FIN, ACK] Seq=408 Ack=401 Win=131392 Len=0 TSval=677674248 TSecr=84951333
435	21.412201	192.168.0.184	192.168.0.243	TCP	66	55603 → 56856 [ACK] Seq=401 Ack=409 Win=65024 Len=0 TSval=84951349 TSecr=677674248
436	21.448320	192.168.0.243	192.168.0.184	TCP	78	56857 → 55603 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=82452558 TSecr=0 SACK_PERM
437	21.448560	192.168.0.184	192.168.0.243	TCP	74	55603 → 56857 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=84951386 TSecr=82452558
438	21.458509	192.168.0.243	192.168.0.184	TCP	66	56857 → 55603 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=82452567 TSecr=84951386
439	21.458509	192.168.0.243	192.168.0.184	HTTP	432	GET /favicon.ico HTTP/1.1
440	21.459113	192.168.0.184	192.168.0.243	HTTP	159	HTTP/1.1 404 Not Found (text/html)
441	21.459205	192.168.0.184	192.168.0.243	TCP	66	55603 → 56857 [FIN, ACK] Seq=94 Ack=367 Win=65024 Len=0 TSval=84951396 TSecr=82452567
442	21.466419	192.168.0.243	192.168.0.184	TCP	66	56857 → 55603 [ACK] Seq=367 Ack=95 Win=131712 Len=0 TSval=82452576 TSecr=84951396
443	21.467567	192.168.0.243	192.168.0.184	TCP	66	56857 → 55603 [FIN, ACK] Seq=367 Ack=95 Win=131712 Len=0 TSval=82452576 TSecr=84951396
444	21.467621	192.168.0.184	192.168.0.243	TCP	66	55603 → 56857 [ACK] Seq=95 Ack=368 Win=65024 Len=0 TSval=84951405 TSecr=82452576

En la traza se muestra un intento fallido de autenticación: el cliente envía un POST /autenticacion con el campo contra y el servidor responde inmediatamente con un HTTP 403 Forbidden. Esta situación fue provocada para observar cómo se refleja un error de autenticación en Wireshark. Al inspeccionar el paquete se ve claramente el valor enviado en el formulario y la respuesta de rechazo del servidor. El servidor corta el flujo antes de permitir el upload mostrando cómo se comporta el sistema frente a credenciales incorrectas.

648	33.698134	192.168.0.243	192.168.0.184	TCP	78	56858 → 55603	[SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3463081867 TSecr=0 SACK_PERM
649	33.698403	192.168.0.184	192.168.0.243	TCP	74	55603 → 56858	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=84963635 TSecr=3463081867
650	33.702995	192.168.0.243	192.168.0.184	TCP	66	56858 → 55603	[ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=3463081879 TSecr=84963635
651	33.702995	192.168.0.243	192.168.0.184	TCP	630	56858 → 55603	[PSH, ACK] Seq=1 Ack=1 Win=131776 Len=564 TSval=3463081879 TSecr=84963635 [TCP PDU reassembled in 654]
652	33.703782	192.168.0.184	192.168.0.243	HTTP	183		HTTP/1.1 403 Forbidden (text/html)
653	33.703866	192.168.0.184	192.168.0.243	TCP	66	55603 → 56858	[FIN, ACK] Seq=118 Ack=565 Win=64768 Len=0 TSval=84963641 TSecr=3463081879
654	33.704065	192.168.0.243	192.168.0.184	HTTP	81		POST /autenticacion HTTP/1.1 (application/x-www-form-urlencoded)
655	33.704161	192.168.0.184	192.168.0.243	TCP	54	55603 → 56858	[RST, ACK] Seq=119 Ack=580 Win=0 Len=0
656	33.707919	192.168.0.243	192.168.0.184	TCP	66	56858 → 55603	[ACK] Seq=580 Ack=119 Win=131712 Len=0 TSval=3463081884 TSecr=84963641
657	33.708890	192.168.0.243	192.168.0.184	TCP	66	56858 → 55603	[FIN, ACK] Seq=580 Ack=119 Win=131712 Len=0 TSval=3463081885 TSecr=84963641

Una vez superado el intento fallido y enviada la contraseña correcta, el navegador repite el POST /autenticacion y el servidor responde con HTTP/1.1 200 OK, habilitando la interfaz de subida. Inmediatamente después, se abre una nueva conexión TCP para el envío del archivo. En Wireshark, este momento aparece claramente marcado por el POST /upload, acompañado ahora por los headers característicos de una transferencia según el tipo de archivo (por ejemplo, image/jpeg o image/png).

23…	79.787155	192.168.0.243	192.168.0.184	TCP	78	56863 → 55603	[SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=1398758192 TSecr=0 SACK_PERM
23…	79.787313	192.168.0.184	192.168.0.243	TCP	74	55603 → 56863	[SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=85009724 TSecr=1398758192
23…	79.791109	192.168.0.243	192.168.0.184	TCP	66	56863 → 55603	[ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=1398758200 TSecr=85009724
23…	79.791109	192.168.0.243	192.168.0.184	TCP	674	56863 → 55603	[PSH, ACK] Seq=1 Ack=1 Win=131776 Len=608 TSval=1398758200 TSecr=85009724 [TCP PDU reassembled in 2347]
23…	79.793239	192.168.0.243	192.168.0.184	TCP	58…	56863 → 55603	[ACK] Seq=609 Ack=1 Win=131776 Len=5792 TSval=1398758202 TSecr=85009724 [TCP PDU reassembled in 2347]
23…	79.793314	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=1 Ack=6401 Win=65280 Len=0 TSval=85009731 TSecr=1398758200
23…	79.794270	192.168.0.243	192.168.0.184	TCP	15…	56863 → 55603	[ACK] Seq=6401 Ack=1 Win=131776 Len=1448 TSval=1398758202 TSecr=85009724 [TCP PDU reassembled in 2347]
23…	79.794308	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=1 Ack=7849 Win=65280 Len=0 TSval=85009732 TSecr=1398758202
23…	79.794351	192.168.0.243	192.168.0.184	TCP	58…	56863 → 55603	[ACK] Seq=7849 Ack=1 Win=131776 Len=5792 TSval=1398758202 TSecr=85009724 [TCP PDU reassembled in 2347]
23…	79.794388	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=1 Ack=13641 Win=65280 Len=0 TSval=85009732 TSecr=1398758202
23…	79.797476	192.168.0.243	192.168.0.184	TCP	58…	56863 → 55603	[ACK] Seq=13641 Ack=1 Win=131776 Len=5792 TSval=1398758206 TSecr=85009731 [TCP PDU reassembled in 2347]
23…	79.797549	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=1 Ack=19433 Win=65280 Len=0 TSval=85009735 TSecr=1398758206
23…	79.798617	192.168.0.243	192.168.0.184	TCP	73…	56863 → 55603	[ACK] Seq=19433 Ack=1 Win=131776 Len=7240 TSval=1398758206 TSecr=85009731 [TCP PDU reassembled in 2347]
23…	79.798677	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=1 Ack=26673 Win=65280 Len=0 TSval=85009736 TSecr=1398758206
23…	79.798968	192.168.0.243	192.168.0.184	TCP	58…	56863 → 55603	[ACK] Seq=26673 Ack=1 Win=131776 Len=5792 TSval=1398758206 TSecr=85009732 [TCP PDU reassembled in 2347]
23…	79.798968	192.168.0.243	192.168.0.184	HTTP	299		POST /upload HTTP/1.1 (JPEG JFIF image)
23…	79.799024	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=1 Ack=32698 Win=65280 Len=0 TSval=85009736 TSecr=1398758206
23…	79.800415	192.168.0.184	192.168.0.243	HTTP	184		HTTP/1.1 200 OK (text/html)
23…	79.800479	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[FIN, ACK] Seq=119 Ack=32698 Win=65280 Len=0 TSval=85009738 TSecr=1398758206
23…	79.803747	192.168.0.243	192.168.0.184	TCP	66	56863 → 55603	[ACK] Seq=32698 Ack=120 Win=131712 Len=0 TSval=1398758213 TSecr=85009738
23…	79.805129	192.168.0.243	192.168.0.184	TCP	66	56863 → 55603	[FIN, ACK] Seq=32698 Ack=120 Win=131712 Len=0 TSval=1398758214 TSecr=85009738
23…	79.805199	192.168.0.184	192.168.0.243	TCP	66	55603 → 56863	[ACK] Seq=120 Ack=32699 Win=65280 Len=0 TSval=85009742 TSecr=1398758214

A partir de ese punto, el archivo comienza a ser transmitido como un flujo continuo de datos que la capa de transporte divide automáticamente en múltiples segmentos TCP. Cada uno aparece en Wireshark etiquetado como TCP segment of a reassembled PDU, indicando que pertenece al cuerpo del POST y que será reconstruido por el receptor antes de entregarlo a la capa de aplicación.

Cuando el servidor termina de recibir todos los fragmentos y reconstruye el archivo completo, envía una respuesta HTTP/1.1 200 OK indicando que la subida fue exitosa. Tras ello, la comunicación entra en su última fase. Finalmente, el servidor envía un FIN, ACK y el cliente responde con el ACK correspondiente, completando así la transferencia de subida de manera ordenada.

Nos resultó interesante notar que nuestro código de start_server recibe el body del request en bloques de 4096 bytes (debido a la línea “body += client_socket.recv(4096)”), pero por ejemplo, en el segmento con número de secuencia 19433 se llegan a enviar hasta 7240 bytes. Esto se debe a que la longitud de cada segmento está determinada por la comunicación entre los dispositivos en la capa de transporte. La restricción a 4096 bytes incluida en nuestro código solo tiene que ver con la comunicación entre la capa de aplicación y la capa de transporte del mismo servidor, y por lo tanto, no afecta en absoluto a la longitud de los segmentos que los dispositivos se envían entre sí. Esto nos sirvió para evidenciar el rol de intermediario que tiene el protocolo de transporte entre lo que envía el cliente y lo que “ve” o recibe la capa de aplicación del servidor.

Asimismo, en cuanto a la longitud de los paquetes, también observamos que durante la descarga los segmentos se mantuvieron prácticamente constantes (en torno a los 1448 bytes), mientras que en la subida el tamaño de los segmentos varió considerablemente (con valores entre 608 y 7240 bytes). Entendemos que esta diferencia se relaciona con el control de flujo de TCP y con las capacidades distintas de cada dispositivo: el teléfono celular parece requerir recibir datos en segmentos más pequeños (su *receive window* es más pequeña), mientras que la computadora es capaz de recibir flujos

más grandes. Consideramos que este comportamiento se explica mejor por mecanismos de control de flujo que por control de congestión, ya que ambas transferencias tuvieron lugar en la misma red y en el mismo período de tiempo, sin indicios de variaciones en las condiciones del tráfico.

Conclusión

La implementación de este servidor constituyó una instancia valiosa para ejercitar y consolidar muchos de los conceptos centrales de la materia, permitiéndonos observar el funcionamiento real de muchos de los mecanismos que estudiamos teóricamente. El proyecto requirió que interactuaramos directamente con uno de los protocolos de la capa de aplicación, diseñando y manipulando headers HTTP, manejando rutas, interpretando métodos, procesando formularios multipart y organizando la lógica necesaria para gestionar cargas y descargas de archivos.

El trabajo también nos sirvió para ver de primera mano ciertas limitaciones y vulnerabilidades inherentes a nuestro servidor. Al no contar con TLS ni verificación de integridad, todo lo transmitido viaja en texto plano y es fácilmente interpretable. Estos aspectos nos permitieron analizar concretamente cuestiones de seguridad y confidencialidad; conectando directamente los resultados obtenidos con los riesgos reales asociados al diseño de sistemas en red.

Además, problemas prácticos como el manejo de boundaries en multipart/form-data, la negociación de *gzip*, las diferencias de comportamiento entre archivos comprimidos y no comprimidos, y la sensibilidad a errores de autenticación y estructura de request nos pusieron frente a desafíos que requirieron aplicar de manera integrada los contenidos de la materia. A la vez, indagamos y profundizamos por nuestra cuenta, ejercitando la búsqueda de recursos online. Siempre comenzamos consultando la documentación provista, que a su vez disparaba nuevas preguntas que buscamos contestar leyendo páginas oficiales, pero también foros y blogs instructivos.

Finalmente, el trabajo no solo nos permitió ejecutar las funcionalidades pedidas, sino que significó un marco para comprender de forma tangible cómo se articulan los conceptos de redes de computadoras en un sistema real.